

An Autonomous Path Planner Implemented on the Rocky7 Prototype Microrover

Sharon L. Laubach
Jet Propulsion Laboratory
California Institute of Technology
198-219, 4800 Oak Grove Drive
Pasadena, CA 91109
Sharon.Laubach@jpl.nasa.gov

Joel Burdick
Dept. of Mechanical Engineering
California Institute of Technology
Mail Code 104-44
Pasadena, CA 91125
jwb@robby.caltech.edu

Larry Matthies
Jet Propulsion Laboratory
California Institute of Technology
107, 4800 Oak Grove Drive
Pasadena, CA 91109
Larry.Matthies@jpl.nasa.gov

Abstract:

Much prior work in mobile robot path planning has been based on assumptions that are unrealistic for exploration of planetary terrains. Based on the first author's experience with the Mars Pathfinder mission, this paper reviews issues that are critical for successful autonomous navigation of planetary rovers. No currently proposed methodology accurately addresses all of these issues. We next report on an extension of the recently proposed "Tangent Bug" algorithm. The implementation of this extended algorithm on the Rocky 7 Mars Rover prototype at the Jet Propulsion Laboratory is described, and experimental results are presented. In addition, limitations encountered by the Sojourner rover in actual Martian terrain suggest that terrain traversability is a key issue for future interplanetary rover autonomous planning algorithms.

1. Introduction

In the absence of an interplanetary manned space program, planetary scientists must rely upon remote sensing techniques to gather data. An invaluable part of their arsenal is spacecraft which can study the region of interest in detail. Landers in particular are able to bring sensors and instruments into close contact with the planetary environment, in order to study soil mechanics and elemental composition. Landers are also able to image surface features in greater detail than is possible with orbiters or Earth-bound instruments. Landers, however, are limited to a single site for study. A mobile robot which can rove over a larger terrain segment and can carry imagers and instruments is clearly an important addition. The recent Mars Pathfinder mission demonstrated the utility of such an approach: the Sojourner rover carried by the lander to the surface of Mars ranged over an area approximately 20 meters square. The Sojourner rover conducted soil experiments in several different terrains, took detailed images of rocks and soils from centimeters away, and placed its on-board spectrometer on 16 distinct targets (9 rocks and 7 soil sites) [8]. Future missions plan to incorporate mo-

bile robots ("rovers") which are capable of traversing much larger distances, carrying their instruments to a wider variety of features and even caching samples along the way. A key requirement for these new planetary robots is greater navigational autonomy since longer distances must be covered between opportunities to communicate with Earth than in prior missions.

1.1 The "rover problem"

The "rover problem" is this: Given an unknown, rough terrain, the mobile robot must autonomously navigate from its current position to the goal. The rover uses its sensors to determine the nature of its local environment, and plans its path accordingly. Rather than addressing all of the issues which arise in this complex problem, this paper focusses on the aspects relevant to autonomous path planning.

1.2 Relevant work

Much prior work in mobile robot path planning has assumed indoor environments and omnidirectional motion and sensing for the robot. The classical path planners described in Latombe [5] further assume a completely known environment. The classical planners have the useful properties of correctness and completeness. That is, the paths produced by these planners lie completely in the free space of the robot, and the planners will generate a path if one is possible, or will halt otherwise. The class of heuristic planners, such as Brooks' subsumption architecture [1] or the track arbitration schemes developed at CMU [4], as well as the "Go To Waypoint" algorithm employed by the Sojourner rover [8], share the useful property of being able to be made sensor-based much more easily than the classical planners, and can be applied to unknown terrains. Although they are designed to work well in "most" environments, they lack completeness: there is no guarantee that the algorithm will halt, or that the robot will be able to find the goal even if a path exists. The heuristic planners also tend to produce lengthy

paths in natural terrain (relative to the optimal path), as has been seen in field trials with a JPL prototype rover, Rocky7, using a similar "Go To Waypoint" algorithm [6]. (Most traverses of the Sojourner rover on Mars have not been long enough to demonstrate this effect.) It is desirable, then, to develop a path planner which combines the best of both worlds: good local properties with a global convergence criterion. Such planners might be called "complete and correct" sensor-based motion planners. These algorithms are incremental in nature: the robot senses its immediate environment, and determines the "best" local path segment based upon these measurements. After moving along the local path, the robot begins the cycle again.

Two distinct approaches to such planners have been explored, both of which adapt classical methods to a local "visible area": methods which build "roadmaps" within the free space in the visible area, such as Choset's HGVG [2] and the "TangentBug" algorithm of Kamon, et al. [3]; and those which utilise approximate cell decomposition, filling in a grid-based world model incrementally, such as Stentz' D* algorithm. [11] Both D* and TangentBug produce "locally optimal" paths (i.e., the paths are the shortest possible using solely local information). D* has the advantage of being well-developed and of being suited to rough terrain navigation. However, the grid-based world model requires a significant amount of memory for storage. In particular, the completeness of D* depends entirely upon precision of its world model, which is determined by cell granularity. It would be valuable to develop an alternate complete planner which produces locally optimal paths through rough terrain without requiring an extensive grid-based world model.

Based on experience with the Sojourner rover (and the Rocky7 system) we believe that a substantially improved algorithm will be necessary to fulfill the objectives of future planetary missions. The approach taken in this paper is to extend the "TangentBug" algorithm developed by Kamon, Rivlin, and Rimon [3] to be able to account for characteristics of a planetary rover operating in rough terrain. The next section provides a brief review of the original TangentBug algorithm, followed by a discussion of the issues inherent in adapting the algorithm to a "real world" environment. Section 3 addresses some of these issues in a theoretical framework. Section 4 details the implementation of an extended TangentBug on the Rocky7 prototype microrover developed at the Jet Propulsion Laboratory (JPL), and section 5 presents experimental results from this implementation. Section 6 discusses a proposed extension of the current implementation to deal with the important concept of traversability. More generalised obstacle descriptions are an important research is-

sue, particularly as path planners are extended to natural terrains. Section 7 contains concluding remarks.

2. TangentBug

The TangentBug algorithm [3] "sensorises" the classical planner based upon the tangent graph [4]. The classical version assumes complete knowledge of the environment, and guarantees to produce the shortest possible path, if a path exists. TangentBug is based on the idea of "local tangent graphs" which are contained within the area visible to the robot's sensors. These local graphs are then searched for the locally shortest path. In order to maintain the classical planner's completeness, a global criterion is added in order to guarantee convergence to the goal. This criterion, and how it is applied within the algorithm, is discussed in the sequel.

2.1 The Original TangentBug Algorithm

The TangentBug algorithm as originally developed by Kamon, Rivlin, and Rimon [3] is designed for a point robot, with omnidirectional motion and sensing capabilities, in a two-dimensional indoor environment (i.e. flat floors, with obstacles which block both motion and sensing). The limited sensor range of the robot is modelled by a "visibility polygon" (Fig. 1), a star-shaped set which encompasses all points in free space which lie within a radius R of the robot and which can be connected to the robot by a ray lying entirely within the closure of free space. The local tangent graph (LTG) is constructed as follows: the nodes of the graph consist of the current robot position as well as the endpoints of the intersection of the boundaries of sensed obstacles with the circle of radius R around the robot, denoted $B(x,R)$. An additional node is created where the ray connecting the current robot position and the goal intersects $B(x,R)$, as long as the intersection point lies within the closure of the visibility polygon. In effect, this action "projects" the goal position onto the visibility polygon. The edges of the LTG lie wholly within the closure of free space, and are tangent to any obstacles encountered.

The global criterion mentioned earlier is the Euclidean distance, $d(x,G)$, between the current robot position and the goal, measured without regard to blocking obstacles. The algorithm consists of two behaviours: motion-to-goal, and boundary-following. The robot begins with motion-to-goal behaviour, in which the LTG is constructed and searched for the optimal local path to the goal position. Outside of the current visibility polygon, the environment is assumed to consist entirely of free space. In fact, the algorithm does not search the entire LTG, but

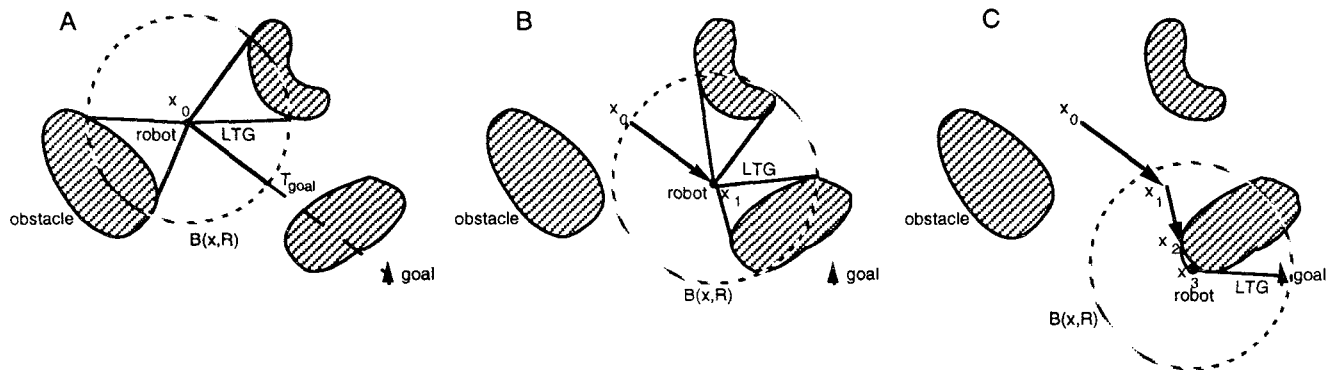


Figure 1. Three stages of the TangentBug algorithm. A) The robot begins at x_0 , computes the LTG, then moves along the locally optimal direction, which brings the robot to x_1 . B) The LTG at x_1 . C) The robot has moved to x_3 by skirting around the obstacle boundary, and the LTG now contains the goal.

rather the subset $G1 = \{V \in \text{LTG} \mid d(V,G) \leq \min(d(x,G), d_{\text{Leave}})\}$, where V denotes a node of the LTG and d_{Leave} is a bookmarking parameter initialised at $d(\text{start},G)$. The resultant local path is thus guaranteed to bring the robot closer to the goal. After moving along the local path, the robot re-senses its environment and begins the cycle again.

The boundary-following behaviour is invoked whenever the robot arrives at a local minimum, i.e. $G1$ is empty. The algorithm initialises $d_{\min}(x,G)$, then chooses a direction to follow around the obstacle boundary and continues in this direction, using $G1$ to construct local shortcuts and recording $d_{\min}(x,G)$, until either the “leaving condition” is met or the robot detects that it has circumnavigated the obstacle. If a loop is detected, the algorithm halts: the goal is not reachable, and in fact lies within the sensed obstacle. The “leaving condition” is as follows: $\exists V \in \text{LTG}$ such that $d(V,G) \leq d_{\min}(x,G)$. Once this condition is met, the robot resets d_{Leave} to $d(V,G)$, and resumes motion-to-goal behaviour. Convergence is thus guaranteed: during motion-to-goal behaviour, $d(x,G)$ decreases monotonically; during boundary-following, the robot leaves the obstacle boundary only when there exists a node in the LTG which lies closer to the goal than any point on the obstacle boundary. In addition, the path length is finite, assuming finite obstacle boundaries, since there are a finite number of both motion-to-goal and boundary-following segments, each of which terminates after finite length.

2.2 Adapting TangentBug to the real world

Though TangentBug incorporates both good local properties—generating locally optimal paths from sensory input—and global convergence to the goal, several thorny issues arise when the algorithm is applied to planetary rovers. It should be stressed that these issues apply to many other path-planners. Obviously, rovers are not

point robots with omnidirectional capabilities. Further, the algorithm assumes an ideal dead-reckoning ability in order to recognise both the goal position and to determine whether the rover has circumnavigated an obstacle. Yet, as has been demonstrated with the Sojourner rover, it is difficult to maintain an accurate estimate of rover position within a global coordinate frame without regular updates from operators on Earth. [8] (There is no GPS on Mars.) In addition, TangentBug assumes omnidirectional sensing capabilities, which is not always practical for planetary rovers. Sojourner’s laser striping system, for example, detects obstacles only in the forward direction.

Other difficulties arise from the environment assumed by the TangentBug algorithm. In the typical terrain encountered in the martian environment (Fig. 2), there exist obstacles which block motion, but not sensing (i.e. the rover has the ability to “see over” many obstacles blocking its path)—thus, the visibility polygon is not strictly a star-shaped set. It may also be desirable to define the goal in terms other than a coordinate point. Alternative goal designations may include a given directional heading with a distance and/or time limit; a specified terrain feature; or a motion termination condition derived

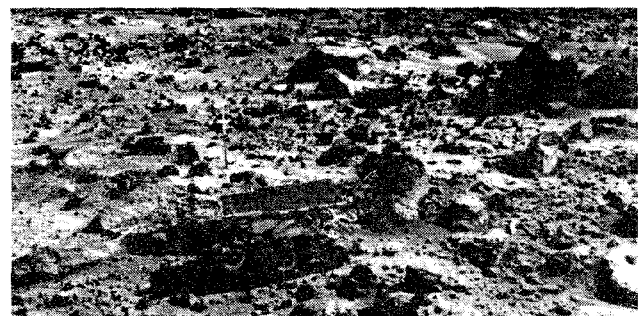


Figure 2. Typical terrain encountered on Mars by the Sojourner rover. The rover is 68 cm long by 48 cm wide, and stands 28 cm tall.

from data sensed by the rover's science instrument payload during traverse. Furthermore, the current algorithm assumes a binary obstacle model: a point in the environment is either an obstacle or lies in free space. It is very desirable to incorporate a notion of traversability, in which certain areas could be designated as "soft" obstacles which could be traversed if need be. Thus, the rover could avoid taking an extremely long route--or worse, declaring a goal position unreachable--when it could have climbed over a rough area it would normally have avoided to reach the goal.

Lastly, TangentBug poses other considerations. While TangentBug produces locally optimal paths, these paths skirt the obstacles encountered--a safety hazard for a rover whose dead-reckoning ability is less than ideal. Also, the boundary-following behaviour is difficult to implement with the forward-looking stereo vision sensors that are used on the microrovers developed at JPL. Finally, work must be done to ensure that the details of implementation do not invalidate the conditions of convergence from the original algorithm.

The TangentBug algorithm is a very promising starting point for the development of path-planners for planetary rovers, because it produces locally-optimal paths using a limited world model and has a global convergence property in an idealised world. The actual application of the planner to planetary rovers, in turn, brings to light considerations and open research questions, which are partially addressed in the next sections.

3. WedgeBug

One of the key issues for implementing a planner on a rover is the limited field-of-view (FOV) of its sensors. In order to address this problem, we have considered a variant of TangentBug, called "WedgeBug", which assumes an FOV with angular size 2α and radius R . The sketch of the proof of completeness of WedgeBug follows: During motion-to-goal, the center of the FOV is pointed in the direction of the goal, and $G1$ is computed within the visible wedge. The motion-to-goal segment ends either when $G1$ is empty--i.e., the robot has encountered a local minimum--or at the goal. If $G1$ is empty, the robot switches to boundary following, in which case the robot takes two views: toward the goal, and tangent to the obstacle boundary. The robot utilises these two views in the same manner as TangentBug's omnidirectional view to skirt the obstacle boundary until either the robot detects a loop--the goal is unreachable--or the leaving condition holds. Both types of segments have finite length. The limited FOV impacts the planner by potentially requiring more use of boundary-following or sliding segments, for obstacles too large to be viewed in a single wedge.

Another issue is the fact that rovers are not point

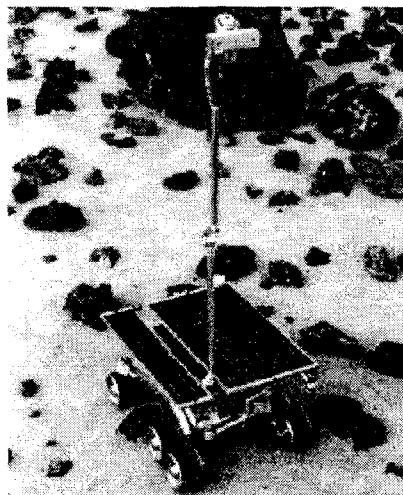


Figure 3. The Rocky7 prototype microrover.

robots. We are exploring the extension of the planner to $SE(2)$, but as an intermediate step we are considering the "silhouettes" of $SE(2)$ obstacles projected onto $\mathcal{R}^2$. The resultant planner--which can now plan as for a point robot-- is not complete, though it is correct.

4. Implementation on the Rocky7 Microrover

An extended version of the TangentBug algorithm described above, called "RoverBug", is being implemented on the JPL Rocky7 prototype microrover (Fig. 3), which is a research vehicle to test technologies for future planetary missions [12]. To the authors' knowledge, this is the first implementation of TangentBug on an actual vehicle. The rover is 60 cm long by 40 cm wide, and stands 33 cm tall, roughly the same size as the Sojourner rover currently on Mars, and carries several science instruments as well as an arm with clamshell scoops for sampling operations. Its mobility chassis consists of a rocker-bogie system able to surmount obstacles $1\frac{1}{2}$ wheel diameters in height (1 wheel diameter = 13 cm). For navigation, it features three pairs of stereo cameras--two mounted below the solar panel (one each front and rear), and one on the deployable 1.2 m mast--as well as a sun sensor for absolute heading measurements and odometry sensors. In addition, a localisation algorithm has recently been developed which utilises mast imagery to aid the rover's dead-reckoning ability [9].

The path-planner is implemented on this system as several functions which are executed sequentially. The general scenario is as follows: the rover is situated in rough terrain, with its mast deployed. The operator views a set of panoramic imagery returned by the rover, and designates a goal position. The operator then sets in motion an autonomous loop: the rover looks toward the goal position with its mast cameras, generates a range image

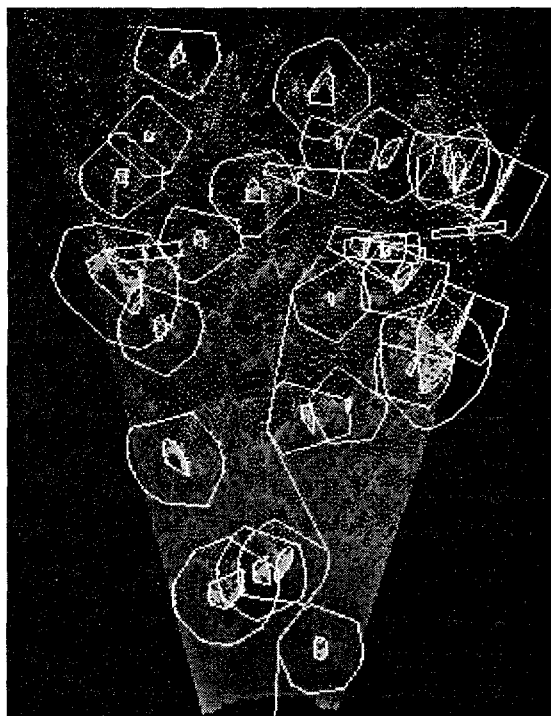


Figure 4. A typical run of RoverBug using stereo data from the JPL "MarsYard". The plot is the range image generated from stereo imagery; overlays depict salient features. Pixels deemed obstacles are dark grey. Each obstacle is enclosed by a white convex hull and a light grey "silhouette". The goal is near the center of the wedge; the autonomously generated path is grey.

from the single-wedge stereo imagery, and begins the path planning calculations. The planner first detects obstacles within the range image, segments out distinct obstacles, and computes their convex hulls. This step relies upon the obstacle detection algorithm presented in [7]. It utilises a simple model of an obstacle--a combination of a step in height and a slope--in order to determine which range image features depict obstacles. The obstacle convex hulls just computed are then passed to a second function which modifies the obstacle boundaries to reflect the "silhouettes" of the obstacles, and to include an empirical safety buffer. The modified convex hulls' vertices are tagged according to whether they now lie within the visible wedge, so the rover will not venture outside of known space. In addition, the goal position is projected onto the boundary of the visible region and added as a single node to the list of convex hull vertices.

The new obstacle boundaries, as well as the goal position, are then passed to the final function, which actually computes the locally optimal path, using a modified A* search [10] on the local tangent graph. A graph search mechanism is required for this implementation due to the fact that the visibility polygon is not strictly star-shaped,

that is, the rover is able to see behind many motion-blocking obstacles. Thus, the resultant LTG is richer than the version in the original TangentBug algorithm. In the case that no path is found, a path to the local minimum closest to the goal is given instead, along with an *ad hoc* target for a "boundary following" stopgap measure: the "Go-To-Waypoint" algorithm already resident on the rover is used to try to skirt the obstacle boundary for a short distance before motion-to-goal is again attempted.

Finally, the planner returns a sequence of waypoints designating the generated path segment, which is followed by an on-board path executor before the rover position is measured (using the localisation algorithm), and the loop is repeated until the goal is attained. Note that the planner can be used at varying levels of autonomy: it could be used either to plan a "strawman" path which could be modified by the user before execution by the rover, or as a fully autonomous single-command system as just described. Technical considerations complicate implementation of the boundary-following behaviour: the only sensor on board which can sense obstacles next to the rover is the mast camera pair, which can only be deployed while the rover is stopped. However, the combination of the expected terrain and the fact that the visible polygon is not strictly a star-shaped set (and thus the LTG is richer) suggests that the boundary following behaviour would not be invoked often.

5. Experimental Results

The "RoverBug" algorithm described above has been tested in several configurations. First, it was tested as a single-step planner--i.e., the start and goal positions were contained in a single panoramic wedge--using imagery collected in the "MarsYard", an outdoor testing facility at JPL designed to mimic the rock distributions seen by the Viking landers on Mars. The functions were implemented in C and run on a Sparc 10 workstation. A typical path generated in this manner is shown in Fig. 4. Next, the algorithm was tested with Rocky7: the planner was used as a single-step, off-line planner which was then executed by the rover: the rover generated the stereo imagery and the range image, which were then used by the off-line C functions to generate a sequence of waypoints which were then used to command Rocky7. These tests revealed unfortunate interaction between the RoverBug planner and Rocky7's current waypoint-based heuristic path executor (the "Go To Waypoint" algorithm described in [12]), indicating that a more appropriate path executor is required.

6. Future Work -- Traversability

Much work remains before RoverBug can be used as a planner for long-range (several hundred meter) traverses

through planetary terrain. Among the most important issues to be addressed is a concept of traversability, to better be able to utilise the rover's mobility characteristics.

As can be seen in the example in Fig. 4, a string of rocks across the far edge of the stereo wedge view has blocked off any progress in this direction. Although it may be possible that a clear path can be found by looking either to the left or right, it may also be true that it may be desirable for the rover to simply drive over the rocks blocking its path. It is not always desirable to have the rover run roughshod over rocks blocking its path. The current model for obstacles sensed by the stereo vision system is a step and slope model: objects exceeding a certain height and slope threshold are deemed obstacles. There is a need for an analysis of traversability issues and movement away from this simple binary notion of obstacles. Indeed, our experience with driving the Sojourner rover on Mars has revealed many cases when a binary obstacle model has resulted in halted motion, often leaving the rover in an undesirable configuration.

Two approaches are being explored to extend TangentBug to allow for these more generalised obstacles. Both involve defining the obstacles as a set of weighted regions, rather like a "potential field" approach, where the weights reflect the "costs" associated with traversing the given region. For example, define a function $\alpha(q, q)$ which assigns a cost to each configuration, based upon the properties of the terrain and mobility characteristics of the robot. The first approach calls the planner repeatedly for each step, where for each iteration the cost threshold associated with what is considered "free space" is raised until either a path is found or the threshold cannot be raised further without increasing the risk to the rover to undesirable levels. The second approach utilises the TangentBug planner once for each step, using as obstacles only those regions marked as "untraversable", and then warps the generated local path according to $\alpha(q, q)$, using calculus of variations techniques, to produce a path which minimises its associated cost.

7. Conclusion

The field of planetary exploration offers a rich environment for mobile robotics research, containing as it does many complicated issues which must be tackled in order to produce successful missions. In particular, this paper addresses some of the issues related to autonomous path-planning for planetary rovers designed to traverse hundreds of meters between uplink opportunities. A sensor-based adaptation of a classical planner with desirable properties has been implemented and tested on the Rocky7 prototype microrover, and several extensions to the current implementation have been proposed, including the vital

area of incorporating traversability into planning decisions. With these tools, mobile robots will prove to be an even more useful and robust addition to the techniques available for planetary exploration.

Acknowledgements The work described here was carried out at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration. We would like to acknowledge the Long Range Science Rover team and the Mars Pathfinder Microrover Flight Experiment team, for help, inspiration, and flight experience with a rover. The first author would particularly like to thank Samad Hayati, Todd Litwin, Andrew Mishkin, and Rich Petras for their invaluable assistance.

8. References

- [1] R. Brooks and A. Flynn, "A robust layered control system for a mobile robot," *IEEE Trans. Robotics Automat.*, vol. 2, no. 1, 1986.
- [2] H. Choset, Sensor Based Motion Planning: The Hierarchical Generalized Voronoi Graph. Ph.D. thesis, California Inst. of Tech., 1996.
- [3] I. Kamon, E. Rivlin, and E. Rimon, "A New Range-Sensor Based Globally Convergent Navigation Algorithm for Mobile Robots," in *Proc. IEEE Conf. Robotics Automat.*, 1996.
- [4] A.J. Kelly, "RANGER--An Intelligent Predictive Controller for Unmanned Ground Vehicles." The Robotics Institute, Carnegie Mellon, 1994.
- [5] J.-C. Latombe, Robot Motion Planning. Kluwer Academic Publishers, 1991.
- [6] L. Matthies, E. Gat, R. Harrison, B. Wilcox, R. Volpe and T. Litwin, "Mars Microrover Navigation: Performance Evaluation and Enhancement," *Autonomous Robots*, vol. 2, no. 4, 1995.
- [7] L. Matthies and P. Grandjean, "Stochastic Performance Modeling and Evaluation of Obstacle Detectability with Imaging Range Sensors," *IEEE Trans. Robotics Automat.*, vol. 10, no. 6, 1994.
- [8] A. Mishkin, J. Morrison, T. Nguyen, H. Stone and B. Cooper, "Operations and Autonomy of the Mars Pathfinder Microrover," to appear in *Proc. IEEE Aerospace Conf.*, 1998.
- [9] C. Olson and L. Matthies, "Maximum Likelihood Rover Localization by Matching Range Maps," to appear in *Proc. IEEE Conf. Robotics Automat.*, 1998.
- [10] A.L. Rankin, Path Planning and Path Execution Software for an Autonomous Nonholonomic Robot Vehicle. Master's thesis, University of Florida, 1993.
- [11] A. Stentz, "Optimal and Efficient Path Planning for Partially-Known Environments," in *Proc. IEEE Conf. Robotics Automat.*, 1994.
- [12] R. Volpe, J. Balam, T. Ohm and R. Ivlev, "The Rocky 7 Mars Rover Prototype," in *Proc. IEEE/RSJ Conf. Intelligent Robots and Sys.*, 1996.